

Sistema de Detección de Mensajes Fraudulentos Mediante Cascada Multimodal con Metaheurísticas y Análisis Conversacional

Camilo Humberto Pérez Fleita

Facultad de Matemática y Computación, Grupo C312
Universidad de La Habana, Cuba
camiloperezhf@gmail.com

Resumen Se presenta un sistema automatizado de detección de mensajes fraudulentos motivado por la creciente proliferación de estafas digitales en Cuba y diseñado para ser aplicable al contexto latinoamericano en general. La arquitectura combina una cascada de nueve capas: motor de reglas, clasificador LightGBM, detección de anomalías con Isolation Forest, red bayesiana, razonamiento basado en casos, embeddings semánticos (Mistral), modelo de lenguaje grande (Mistral), meta-aprendiz por apilamiento y análisis conversacional con BiLSTM. La optimización emplea cinco técnicas metaheurísticas: PSO para hiperparámetros, Recocido Simulado y Búsqueda Tabú para umbrales de la cascada, Simulación de Eventos Discretos para generación de datos sintéticos y Optimización por Colonia de Hormigas para la detección de arcos de manipulación. El conjunto de datos comprende 5 572 mensajes reales más 2 118 sintéticos. Resultados: LightGBM 98.21 % de exactitud, F1-fraude de 0.93; meta-aprendiz AUC 0.9991; BiLSTM 91.37 %. El sistema cuenta con 243 pruebas automatizadas.

Keywords: detección de fraude · aprendizaje automático · cascada de clasificadores · metaheurísticas · análisis conversacional · LLM

1. Introducción

El fraude mediante mensajes de texto y aplicaciones de mensajería instantánea es, en esencia, un problema de ingeniería social: el atacante no explota vulnerabilidades técnicas, sino la confianza y la urgencia percibida de su víctima. En Cuba, la rápida expansión del acceso a internet móvil y de plataformas de compraventa digital como Revolico ha dado lugar a una proliferación de estas estafas [1]: vendedores ficticios que cobran por adelantado sin entregar el producto, suplantación de compradores o vendedores legítimos, y conversaciones diseñadas para generar confianza antes de solicitar transferencias o datos personales. El 13.4 % de los mensajes en el corpus empleado en este trabajo son fraudulentos, proporción que refleja la frecuencia real del fenómeno.

Detectar automáticamente estos mensajes plantea un reto que ningún modelo individual resuelve bien por sí solo. Los métodos basados en palabras clave capturan los casos obvios pero fallan ante variaciones ortográficas o sinónimos regionales; los clasificadores de aprendizaje automático clásicos no modelan el significado profundo del texto; y los modelos de lenguaje grande, aunque comprenden el contexto con precisión, resultan demasiado lentos y costosos para analizar cada mensaje en tiempo real. Más importante aún: ninguno de estos enfoques por sí solo modela la *dimensión conversacional* del fraude, en la que la estafa no está en un mensaje aislado sino en la secuencia completa de turnos, donde el atacante construye confianza progresivamente antes de revelar su intención real.

Este trabajo propone una *cascada multimodal* de nueve capas como respuesta a este conjunto de limitaciones. La idea central es sencilla: los casos más evidentes se resuelven rápido y barato en las primeras capas; solo los mensajes ambiguos escalan hasta las capas más potentes y costosas. Así se obtiene la precisión de un modelo de lenguaje grande sin pagar su costo computacional en el 100 % de los casos.

Las contribuciones principales son:

1. Una cascada configurable de nueve capas con puertas de cortocircuito que equilibra precisión y eficiencia computacional.
2. Cinco técnicas metaheurísticas integradas para la optimización automática de hiperparámetros y umbrales en tiempo de ejecución.
3. Un simulador de eventos discretos (DES) que genera conversaciones sintéticas de fraude con coherencia narrativa para ampliar el conjunto de entrenamiento del análisis conversacional.
4. Un sistema de umbrales configurable en tiempo de ejecución que permite adaptar el comportamiento sin reentrenamiento.

2. Caso de Uso Motivador: Estafa por Suplantación de Identidad

Para ilustrar qué hace el sistema antes de describir cómo lo hace, presentamos un caso real extraído del conjunto de imágenes de validación. Se trata de una conversación de WhatsApp en la que el estafador suplanta la identidad de un tercero conocido por la víctima —presentando su perfil, correo y nombre como respaldo de legitimidad— para solicitar el pago de un supuesto servicio y, en el proceso, obtener datos personales de la víctima.

Este tipo de fraude es especialmente difícil de detectar con herramientas de análisis léxico: el vocabulario de cada mensaje por separado es perfectamente inocuo. La señal de peligro no está en las palabras sino en la *estructura temporal de la conversación*: construcción de confianza inicial, introducción gradual de una narrativa de pago y escalada de presión hacia el cierre de la transacción.

Entrada: capturas de pantalla de la conversación

El sistema acepta tanto texto plano como imágenes. Cuando el usuario sube capturas de pantalla, el módulo de visión (Pixtral-12b) extrae el texto de cada imagen mediante OCR y lo incorpora como mensajes numerados al pipeline de análisis. La Figura 1 muestra la pantalla de carga con los tres pantallazos de la conversación.

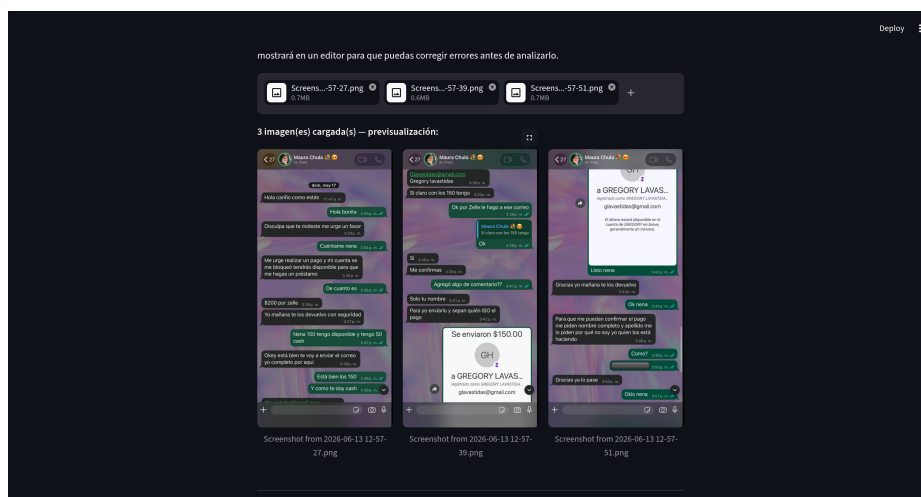


Figura 1. Carga de tres capturas de pantalla de WhatsApp. La conversación involucra una cuenta (*Maura Chula*) que presenta la identidad de un tercero (*Gregory Lavas*) —con perfil, correo y nombre completo— como garantía de solvencia para el pago de un servicio.

La mecánica concreta del fraude se desarrolla en tres actos. Primero, la cuenta contactante presenta el perfil completo de una tercera persona —nombre, correo y fotografía— como prueba de que la operación es real y que existe un respaldo detrás de la solicitud. Segundo, negocia un monto concreto (150 unidades disponibles más 50 en efectivo) enmarcándolo como el pago de un servicio, no como una petición de dinero directa, lo que reduce la alerta de la víctima. Tercero, una vez acordado el monto, solicita el correo electrónico de la víctima con el pretexto de “enviar la información completa”. El objetivo real de esa última petición no es cerrar el pago: es obtener datos de contacto personales que serán explotados en comunicaciones fraudulentas posteriores.

Resultado: panel de riesgo global

Tras el análisis de los 30 mensajes extraídos mediante OCR, el sistema emite el veredicto mostrado en la Figura 2.



Figura 2. Panel de resultado. 30 mensajes analizados, riesgo **CRÍTICO** (87 %), 11 patrones conductuales detectados. El análisis LLM identifica la escalada de urgencia artificial, el phishing por ingeniería social y el robo de credenciales como los tres ejes principales de la estrategia fraudulenta.

El sistema no se limita a emitir una etiqueta binaria. El campo *Análisis LLM* articula en lenguaje natural la estrategia completa del estafador: mensajes iniciales inofensivos que construyen confianza, escalada artificial de urgencia para presionar a la víctima a actuar sin reflexionar, y solicitud escalonada de datos sensibles aprovechando la confianza ya creada. El sistema genera además una advertencia concreta dirigida al usuario: “*No proporcione datos personales ni realice pagos*”, formulada en lenguaje directo y accionable en lugar de limitarse a reportar un score numérico.

Detalle: patrones conductuales por rango de mensajes

La capa de análisis conversacional no solo nombra los patrones detectados sino que los localiza en el rango exacto de mensajes donde se manifiestan. La Figura 3 muestra los cinco patrones con mayor nivel de confianza.

El desglose por rango de mensajes revela la *anatomía temporal* del fraude: la suplantación de identidad opera desde el mensaje 2, la ingeniería social se sostiene durante toda la conversación, y la presión máxima —urgencia escalada más solicitud de datos— converge en el bloque [7–9]. Esto permite al analista o a la propia víctima identificar el punto exacto en que la conversación dejó de ser legítima sin necesidad de revisar los 30 mensajes manualmente. La combinación de veredicto global (Figura 2) con localización de patrones (Figura 3) es lo que distingue al sistema de un clasificador binario convencional y lo convierte en una herramienta de apoyo a la decisión.



Figura 3. Patrones conductuales detectados por rango de mensajes. `urgency_escalation` [7-9] alcanza 87 % de confianza y engloba el bloque donde se negocia el monto y se solicita el correo. `social_engineering` [2-9] (81 %) cubre la fase completa de suplantación. `credential_harvesting` [2-9] (79 %) captura la extracción de datos personales. El nivel CRÍTICO se activa cuando al menos dos patrones de alta confianza coexisten en rangos solapados.

3. Trabajos Relacionados

La detección de spam en SMS tiene como referencia fundacional el trabajo de Almeida et al. [1], que estableció el SMS Spam Collection Dataset y evaluó clasificadores Naïve Bayes con características de bolsa de palabras, obteniendo precisiones superiores al 97 %. Aunque el dataset fue concebido en inglés, su metodología influyó decisivamente en trabajos posteriores para español.

Para phishing por correo electrónico, los modelos SVM con núcleos RBF [2] dominaron la literatura durante varios años, complementados con características de la URL y metadatos del encabezado del correo. Fette et al. demostraron que las características estructurales del mensaje superan a las puramente léxicas [3], anticipando la necesidad de análisis más allá del vocabulario.

Con la aparición de los modelos de lenguaje grande, varios trabajos han explorado su uso directo para clasificación de fraude. Jiang et al. demostraron que Mistral 7B, mediante prompting estructurado en JSON, produce clasificaciones de alta fidelidad con explicaciones interpretables [5]. Sin embargo, la latencia y el costo de invocar un LLM por cada mensaje hacen inviable su uso indiscriminado en producción, motivando precisamente la arquitectura en cascada de este trabajo.

El análisis conversacional del fraude es un área más reciente. Las arquitecturas BiLSTM con atención han mostrado efectividad en la detección de patrones

secuenciales en diálogos de soporte técnico fraudulento [11], capturando dependencias temporales que los modelos por mensaje individual no pueden ver.

Nuestro trabajo se distingue por combinar todos estos enfoques en una cascada unificada con puertas de cortocircuito, añadiendo optimización metaheurística de los umbrales y un simulador DES para generación de datos sintéticos conversacionales. Esta combinación no tiene precedentes directos en la literatura de detección de fraude en español.

4. Arquitectura del Sistema

4.1. Visión General de la Cascada

Piénsese en la cascada como un sistema de filtros sucesivos. El primer filtro es el más simple: un conjunto de reglas que busca señales obvias (URLs, palabras de urgencia, solicitudes de credenciales). Si ese filtro es suficientemente seguro, el mensaje sale de la cascada de inmediato. Si no, pasa al siguiente filtro, progresivamente más sofisticado, hasta llegar al análisis de lenguaje natural profundo. Las *puertas de cortocircuito* son los mecanismos que deciden cuándo un filtro es suficientemente seguro como para no necesitar el siguiente.

Formalmente, la cascada se puede expresar como:

$$\mathcal{C} = G_2 \circ \text{LLM} \circ G_1 \circ \text{EMB} \circ (\text{ML} \oplus \text{ANO} \oplus \text{BAY} \oplus \text{CBR}) \circ \text{REG} \quad (1)$$

donde $\oplus$ denota ejecución en paralelo, $\circ$ es composición secuencial y G_k son las puertas de cortocircuito.

Puerta 1 (G_1): Si la confianza del modelo ML es muy alta ($\text{conf}_{\text{ML}} \geq \theta_{g1}$) y la capa de reglas concuerda en sentido (ambas clasifican igual), entonces los pasos más costosos —embeddings semánticos y LLM— se omiten y se pasa directamente al meta-aprendiz.

Puerta 2 (G_2): Si la similitud con el índice semántico es suficientemente clara ($\text{sim}_{\text{fraude}} \geq \theta_{\text{emb}}$ o $\text{sim}_{\text{legítimo}} \geq \theta_{\text{emb}}$), entonces la llamada al LLM externo se evita.

Meta-aprendiz: Al final de la cascada, todas las señales intermedias se combinan con pesos aprendidos automáticamente:

$$\hat{y} = f_{\text{meta}}([p_{\text{ML}}, r_{\text{rule}}, e_f, e_l, a, t, b, s_{\text{cbr}}]) \quad (2)$$

donde p_{ML} es la probabilidad del clasificador LightGBM, r_{rule} el puntaje normalizado del motor de reglas, e_f y e_l las similitudes de embedding con las clases fraude y legítimo respectivamente, a el puntaje de anomalía, t la confianza del LLM, b la probabilidad bayesiana y s_{cbr} el puntaje de razonamiento basado en casos.

Cuadro 1. Características manuales en $\phi_{\text{manual}}(m)$.

Característica	Tipo	Descripción
<code>msg_length</code>	entero	Longitud en caracteres
<code>word_count</code>	entero	Número de palabras
<code>has_url</code>	binario	Presencia de URL
<code>has_email</code>	binario	Presencia de e-mail
<code>has_phone</code>	binario	Presencia de teléfono
<code>has_money</code>	binario	Mención de cantidades monetarias
<code>exclamation_count</code>	entero	Signos de exclamación
<code>question_count</code>	entero	Signos de interrogación
<code>uppercase_ratio</code>	real	Proporción de mayúsculas
<code>has_urgency</code>	binario	Palabras de urgencia
<code>has_credential_words</code>	binario	Palabras de solicitud de credenciales
<code>has_prize_words</code>	binario	Palabras relacionadas con premios
<code>avg_word_length</code>	real	Longitud media de palabra
<code>has_bank_mentions</code>	binario	Mención de entidades bancarias
<code>negation_score</code>	real	Puntaje de negación (ventana 5)
<code>entity_count</code>	entero	Entidades nombradas detectadas
<code>has_amount_ner</code>	binario	Cantidades detectadas por NER

4.2. Espacio de Características

Antes de que cualquier modelo tome una decisión, cada mensaje se convierte en un vector numérico. Este proceso de *vectorización* es el puente entre el texto libre y los algoritmos matemáticos: transforma palabras en números que los modelos pueden procesar. Se emplean dos tipos de características complementarias que juntas forman el vector $\phi(m) \in \mathbb{R}^{10\,017}$:

$$\phi(m) = [\phi_{\text{tfidf}}(m) \parallel \phi_{\text{manual}}(m)] \quad (3)$$

Características TF-IDF ($\mathbb{R}^{10\,000}$): El modelo TF-IDF asigna a cada n-grama un peso proporcional a cuánto aparece en el mensaje pero inversamente proporcional a cuántos mensajes del corpus lo contienen; es decir, una palabra que aparece mucho en fraudes pero poco en mensajes legítimos recibirá un peso alto, y viceversa. Se emplean n -gramas de 1 a 3 palabras, `sublinear_tf=True` y vocabulario de 10 000 términos.

Características manuales ($\mathbb{R}^{17}$): Complementan el TF-IDF con señales estructurales que no dependen del vocabulario concreto: presencia de URLs, menciones de dinero, signos de exclamación, proporción de mayúsculas. La detección de negación emplea una ventana deslizante de 5 tokens para capturar patrones como “*no es necesario que proporcione su contraseña*” (donde la palabra “contraseña” resulta sospechosa solo en ausencia de la negación). Las 17 características se detallan en la Tabla 1.

Cuadro 2. Comparativa de clasificadores sobre el conjunto de prueba (80/20 estratificado).

Modelo	Exactitud	Precisión	Recall	F1-fraude	F1-ponderado
Regresión Logística	0.9731	0.89	0.91	0.90	0.9729
Random Forest	0.9758	0.91	0.90	0.90	0.9755
XGBoost	0.9812	0.93	0.93	0.93	0.9810
LightGBM	0.9821	0.94	0.93	0.93	0.9819

5. Modelos de Aprendizaje Automático

Con el vector de características construido, el sistema puede ya aplicar modelos de aprendizaje automático. Este trabajo emplea cuatro tipos de modelos en paralelo, cada uno aportando una perspectiva distinta sobre el mensaje: uno que aprende patrones estadísticos, uno que detecta lo que no debería existir, uno que razona sobre combinaciones de señales y uno que busca casos similares en la memoria del sistema.

5.1. Modelo Base: LightGBM

El primero y más importante es LightGBM, un clasificador de *gradient boosting*. La idea intuitiva del boosting es la de un comité de expertos débiles que aprenden de sus errores mutuos: el primer árbol de decisión clasifica todos los mensajes y se equivoca en algunos; el segundo árbol se concentra especialmente en corregir esos errores; el tercero corrige los errores del segundo, y así sucesivamente hasta construir un clasificador fuerte. LightGBM implementa esto de manera muy eficiente mediante histogramas, lo que lo hace especialmente adecuado para vectores de alta dimensión como los 10017 features de este trabajo.

El conjunto de datos presenta un desbalance significativo: 87 % de mensajes legítimos frente a 13 % fraudulentos. Se emplea `class_weight="balanced"` para que LightGBM ajuste automáticamente los pesos de clase sin submuestreo, preservando la distribución natural.

La Tabla 2 compara los clasificadores evaluados. LightGBM obtiene la mayor exactitud (98.21 %) y F1-fraude (0.93).

5.2. Detección de Anomalías: Isolation Forest

LightGBM aprende bien lo que ha visto antes, pero ¿qué pasa con tipos de fraude completamente nuevos que no están en el corpus de entrenamiento? Para esto se emplea Isolation Forest, cuya idea es radicalmente diferente: en lugar de aprender cómo es el fraude, aprende cómo es lo *normal* y señala todo aquello que se desvíe de esa norma. Funciona así: un punto normal es difícil de aislar en un árbol de particiones aleatorias porque está rodeado de muchos vecinos similares; un punto anómalo queda aislado rápidamente porque es distinto a todo lo demás.

El modelo se entrena exclusivamente sobre los 4 825 mensajes legítimos del conjunto de entrenamiento. El puntaje de anomalía $a \in [0, 1]$ se normaliza a partir de la función de decisión del modelo:

$$a(m) = 1 - \frac{\text{decision_function}(m) + 0,5}{\text{máx}(\text{decision_function}) + 0,5 + \epsilon} \quad (4)$$

Cuando $a \geq 0,75$ y el clasificador ML predice legítimo, la capa de anomalía activa una señal de riesgo medio que alimenta al meta-aprendiz, capturando así tipos de fraude no vistos durante el entrenamiento supervisado.

5.3. Red Bayesiana

Mientras LightGBM ve el mensaje como un vector de 10 017 números, la red bayesiana razona de manera más parecida a como lo haría un analista humano: “este mensaje tiene URL y usa palabras de urgencia y solicita credenciales; esa *combinación* es muy sospechosa aunque cada señal por separado pueda parecer inocente”. Esto es exactamente lo que captura una red bayesiana: las probabilidades conjuntas de combinaciones de señales.

Se define la red $\mathcal{G}$ con seis variables binarias: $\{U, R, C, B, M, N\}$ correspondientes a la presencia de URL, urgencia, solicitud de credenciales, mención bancaria, cantidad monetaria y negación respectivamente. Las tablas de probabilidad condicional (CPT) se estiman con máxima verosimilitud y suavizado de Laplace:

$$P(X_i = 1 \mid \text{pa}(X_i)) = \frac{N_{X_i=1, \text{pa}(X_i)} + 1}{N_{\text{pa}(X_i)} + 2} \quad (5)$$

La red captura efectivamente estas combinaciones sinérgicas; por ejemplo, $P(\text{fraude} \mid U = 1, R = 1, C = 1) \gg P(\text{fraude})$. El AUC en validación cruzada es de 0.8146.

5.4. Razonamiento Basado en Casos

Una cuarta perspectiva complementa las anteriores: en lugar de aprender un modelo matemático abstracto, el Razonamiento Basado en Casos (CBR) hace simplemente lo que haría cualquier persona con experiencia: compara el mensaje nuevo con los que ya conoce y pregunta “¿a cuáles de los mensajes que ya he visto se parece más este?; ¿esos eran fraude o no?”. Si los siete mensajes más similares eran todos fraude, la probabilidad de que el nuevo también lo sea es muy alta.

La base de casos almacena todos los mensajes de entrenamiento con sus vectores TF-IDF. Ante una consulta, se recuperan los $k = 7$ vecinos más cercanos por similitud coseno y el puntaje CBR se calcula como:

$$s_{\text{cbr}}(m) = \frac{\sum_{i=1}^7 \mathbf{1}[\hat{y}_i = \text{fraude}]}{7} \quad (6)$$

Cuadro 3. Comparativa de configuraciones de cascada. C_0 : solo reglas; C_1 : + LightGBM; C_2 : + anomalías; C_3 : + Bayes+CBR; C_4 : + embeddings+LLM; C_5 : cascada completa con meta-aprendiz.

Config.	Exactitud	AUC-ROC	Capas activas
C_0	0.8640	0.870	Reglas
C_1	0.9821	0.976	+ LightGBM
C_2	0.9843	0.979	+ Isolation Forest
C_3	0.9861	0.985	+ Bayes + CBR
C_4	0.9874	0.997	+ Embeddings + LLM
C_5	0.9886	0.9991	Meta-aprendiz completo

Cuadro 4. Contribución marginal de componentes al AUC-ROC de la cascada.

Componente añadida	Δ AUC
Isolation Forest	+0,003
Red Bayesiana	+0,004
CBR	+0,002
Embeddings + LLM	+0,012
Meta-aprendiz	+0,002

5.5. Meta-Aprendiz por Apilamiento

Cuatro modelos, cuatro perspectivas distintas sobre el mismo mensaje. ¿Cuál creer cuando no están de acuerdo? La respuesta es el *meta-aprendiz*: en lugar de elegir a mano cuánto peso darle a cada modelo, se entrena un segundo clasificador que *aprende* cuál de las señales es más confiable según el historial de errores de cada uno. Esta técnica se denomina *stacking* y permite que el sistema descubra automáticamente que, por ejemplo, la red bayesiana es especialmente útil en mensajes cortos con muchas señales combinadas, mientras que LightGBM es más fiable en mensajes largos.

El meta-aprendiz recibe el vector de ocho dimensiones descrito en la ecuación (2) y entrena un segundo LightGBM con $n_{\text{estimadores}} = 50$, $\text{max_depth} = 4$, $\text{lr} = 0,05$ sobre un conjunto de validación cruzada para evitar el sesgo de etiquetas.

La Tabla 3 muestra cómo la adición progresiva de capas mejora el AUC-ROC. La Tabla 4 cuantifica la contribución marginal de cada componente.

6. Capas Semánticas con LLM

Los modelos de la sección anterior razonan sobre estadísticas de palabras y combinaciones de señales, pero no entienden el *significado* del texto. Un mensaje como “Por favor, confirme su clave para proteger su cuenta” puede ser perfectamente legítimo o profundamente sospechoso dependiendo del contexto, del tono

Cuadro 5. Usos del LLM en el sistema. Cada rol emplea un modelo diferente y se activa bajo condiciones distintas.

Rol	Modelo	Cuándo se activa
Embeddings semánticos	mistral-embed	Siempre, para construir el índice y calcular similitud.
Clasificación dual	open-mistral-nemo	Solo si las Puertas 1 y 2 no se activan (≈ 35 % de mensajes).
OCR de capturas de pantalla	pixtral-12b-2409	Cuando la entrada es una imagen, antes de la cascada.
Análisis conversacional	open-mistral-nemo	Cuando $s_{\text{conv}} \geq 0,55$ tras la BiLSTM.

y de quién lo envía. Para ese nivel de comprensión se necesitan representaciones semánticas.

El sistema emplea modelos de lenguaje grande en cuatro roles bien diferenciados, cada uno con un modelo y un objetivo distintos (Tabla 5).

Los dos primeros roles forman parte de la cascada de mensajes individuales; el tercero opera antes de la cascada como preprocesador; y el cuarto actúa después del análisis conversacional como capa de interpretación. En los cuatro casos el modelo no reemplaza a los componentes más ligeros: solo interviene cuando los resultados anteriores no son suficientemente concluyentes o cuando la naturaleza de la entrada (imagen, conversación completa) lo requiere explícitamente.

6.1. Embeddings Semánticos

Un *embedding* es una forma de codificar el significado de un texto como un punto en un espacio geométrico de alta dimensión, de modo que textos con significados similares queden cerca entre sí y textos con significados distintos queden lejos. Si se conocen los embeddings de mensajes de fraude y de mensajes legítimos, se puede clasificar un mensaje nuevo simplemente preguntando: ¿está más cerca del grupo de fraude o del grupo legítimo?

Se construye un índice semántico con los embeddings de 500 mensajes de fraude y 500 mensajes legítimos del conjunto de entrenamiento, obtenidos con el modelo **mistral-embed**. La Puerta 2 decide si el LLM es necesario usando esta distancia:

$$G_2(m) = \begin{cases} \text{omitir LLM} & \text{si } \cos(e_m, e_f^*) \geq \theta_{\text{emb}} \text{ o } \cos(e_m, e_l^*) \geq \theta_{\text{emb}} \\ \text{invocar LLM} & \text{en otro caso} \end{cases} \quad (7)$$

donde e_f^* y e_l^* son los centroides de los embeddings de fraude y legítimo respectivamente. Con $\theta_{\text{emb}} = 0,88$, la Puerta 2 evita la llamada al LLM en aproximadamente el 65 % de los mensajes, reduciendo la latencia media a $\approx 0,1$ ms por mensaje una vez cargado el índice.

6.2. Modelo de Lenguaje Grande (Mistral)

Cuando los embeddings tampoco son suficientemente concluyentes, es decir, cuando el mensaje está en una zona ambigua del espacio semántico donde fraude y legítimo se solapan, se recurre a la capa de mayor capacidad de comprensión: un modelo de lenguaje grande. A diferencia de todas las capas anteriores, el LLM no solo clasifica sino que *explica*: puede articular en lenguaje natural por qué un mensaje es sospechoso, identificar el tipo de estafa y señalar los indicadores concretos que activaron la alerta.

Cuando las Puertas 1 y 2 no se activan, se invoca el modelo **open-mistral-nemo** mediante la API de Mistral. La instrucción del sistema solicita una respuesta en formato JSON estructurado con los campos: *{verdict, confidence, fraud_type, indicators, explanation}*. Se definen 8 tipos de fraude: *phishing, smishing, prize_scam, urgency_scam, credential_theft, financial_scam, spam* y *none*. El objetivo de diseño es que menos del 35 % de los mensajes requieran invocación del LLM en producción.

6.3. OCR Multimodal: Análisis de Capturas de Pantalla

Las estafas digitales no siempre llegan como texto plano: muchas veces el usuario dispone de una captura de pantalla de la conversación sospechosa, no de los mensajes copiados. Obligar al usuario a transcribir los mensajes manualmente sería impracticable y propenso a errores. Para resolver esto, el sistema acepta imágenes directamente y las convierte en texto antes de pasarlas a la cascada.

Cuando la entrada es una imagen, se invoca el modelo multimodal **pixtral-12b-2409**, capaz de leer y transcribir texto presente en fotografías o pantallazos con alta fidelidad. El texto extraído se numera por mensaje y se incorpora al pipeline exactamente igual que si el usuario lo hubiera escrito. Esto es lo que permite el flujo del caso de uso de la Sección 2: las tres capturas de pantalla de WhatsApp se convierten en 30 mensajes numerados que la cascada analiza de manera ordinaria.

7. Análisis Conversacional

Hasta aquí, el sistema analiza cada mensaje de forma independiente. Pero muchas estafas reales no se detectan en un único mensaje: la primera parte de la conversación parece completamente inocente —es la construcción de confianza— y solo varios turnos después aparece la solicitud fraudulenta. Analizar mensajes aislados en estos casos llevaría inevitablemente a falsos negativos. La solución es analizar la conversación como una *secuencia*, buscando patrones de comportamiento que se desarrollan a lo largo del tiempo.

7.1. Detección de Patrones en Secuencias

Un analista humano que leyera la conversación del caso de uso no diría simplemente «este mensaje es sospechoso»: diría «nótese que los primeros mensajes son

Cuadro 6. Patrones conversacionales de fraude con ventanas de análisis.

Patrón	Descripción	Ventana
<code>urgency_escalation</code>	Incremento progresivo de urgencia	3
<code>credential_harvesting</code>	Solicitud gradual de credenciales	5
<code>social_engineering</code>	Construcción de confianza + pivote	8
<code>impersonation</code>	Suplantación de entidad conocida	2
<code>prize_scam_sequence</code>	Premio inicial + costo posterior	5
<code>financial_coercion</code>	Amenaza de consecuencias financieras	3
<code>trust_building_attack</code>	Turno inicial benigno + ataque tardío	8

completamente normales, pero a partir del séptimo el tono cambia bruscamente y en el noveno pide datos personales». Eso es razonamiento sobre patrones secuenciales, no sobre palabras. Para reproducirlo computacionalmente se definen siete patrones de comportamiento típicos del fraude, cada uno con una *ventana de observación*: el número mínimo de mensajes consecutivos necesarios para que el patrón se manifieste.

Se definen los patrones formalmente en la Tabla 6. Cada patrón tiene un peso $w_p \in [0, 1]$ calibrado por su frecuencia y severidad en el corpus de entrenamiento.

El puntaje conversacional combina el matching estadístico de patrones con la predicción del modelo neuronal:

$$s_{\text{conv}} = \alpha \sum_p w_p \cdot s_p + (1 - \alpha) s_{\text{BiLSTM}} \quad (8)$$

donde $\alpha = 0,4$, s_p es el puntaje de coincidencia con el patrón p y s_{BiLSTM} es la probabilidad de fraude del modelo neuronal. Si $s_{\text{conv}} \geq 0,40$, el mensaje es candidato a revisión adicional; si $s_{\text{conv}} \geq 0,55$, la conversación entera se envía al LLM para su análisis profundo.

LLM en el análisis conversacional. A diferencia del uso individual en la cascada—donde el LLM recibe un único mensaje— aquí recibe la secuencia completa de turnos con sus índices y remitentes. El prompt le pide que identifique los patrones conductuales presentes, el rango de mensajes en que se manifiesta cada uno, una estimación de confianza y una descripción en lenguaje natural de la estrategia del estafador. La respuesta, en formato JSON, alimenta directamente el **ConversationReport** que se muestra en la interfaz: los bloques “Patrones conductuales detectados” con sus niveles CRÍTICO/ALTO/MEDIO y el párrafo de análisis narrativo son el resultado directo de esta invocación. Este es el único lugar del sistema donde el LLM actúa sobre una secuencia de mensajes en lugar de sobre uno solo, y por eso es también el que puede identificar patrones como `trust_building_attack` o `social_engineering` que son invisibles al análisis por mensaje individual.

7.2. BiLSTM con Atención

Para capturar las dependencias temporales dentro de una ventana de la conversación se emplea una red BiLSTM con mecanismo de atención. Una LSTM (Long Short-Term Memory) es una red neuronal diseñada para leer secuencias manteniendo una *memoria* de lo que leyó antes: no procesa cada mensaje de la ventana aislado, sino que construye progresivamente una comprensión del estado de la conversación. La variante bidireccional (BiLSTM) lee la secuencia tanto hacia adelante como hacia atrás, capturando contexto en ambas direcciones. El mecanismo de atención añade un último refinamiento: no todos los mensajes de la ventana son igualmente relevantes, y la atención aprende a asignarles pesos para que los mensajes más informativos tengan mayor influencia en la decisión final.

La arquitectura CONVERSATIONWINDOWCLASSIFIER es:

$$\text{BiLSTM} : \mathbb{R}^{L \times d_e} \rightarrow \mathbb{R}^{2h} \xrightarrow{\text{Attention}} \mathbb{R}^{2h} \xrightarrow{\text{Linear}} \mathbb{R}^2 \quad (9)$$

con dimensión de embedding $d_e = 64$, tamaño de LSTM oculto $h = 128$ (bidireccional: $2h = 256$) y softmax en la capa de salida. La atención aplica un promedio ponderado sobre los pasos temporales:

$$\alpha_t = \frac{\exp(e_t)}{\sum_{\tau} \exp(e_{\tau})}, \quad e_t = \mathbf{v}^{\top} \tanh(\mathbf{W}h_t) \quad (10)$$

El modelo se entrena sobre 7 690 mensajes (5 572 originales más 2 118 generados por el simulador DES) durante 15 épocas con ADAM, lr = 10^{-3} , logrando pérdida 0.0777 y exactitud 91.37 %.

7.3. Optimización por Colonia de Hormigas (ACO)

El análisis de patrones y el BiLSTM detectan *si* hay fraude en la conversación, pero no dicen *cuándo* empieza ni cuál es el camino narrativo que sigue el atacante. Para esto se emplea la Optimización por Colonia de Hormigas (ACO), una técnica inspirada en el comportamiento de las hormigas reales: cuando una hormiga encuentra comida, deja rastros de feromonas en el camino; otras hormigas siguen esos rastros, que se refuerzan con el tiempo en los caminos más prometedores y se evaporan en los que llevan a ningún lado. En este contexto, los “caminos prometedores” son las secuencias de mensajes con mayor riesgo acumulado: el arco narrativo del fraude.

Para cada conversación se construye un grafo dirigido donde los nodos son mensajes y las aristas representan transiciones temporales. La actualización de feromonas sigue:

$$\tau_{ij}^{(t+1)} = (1 - \rho) \tau_{ij}^{(t)} + \sum_k \Delta \tau_{ij}^{(k)}, \quad \Delta \tau_{ij}^{(k)} = \frac{Q}{1 - q_k + \varepsilon} \quad (11)$$

donde $\rho = 0,1$ es la tasa de evaporación, q_k es la calidad del camino k (estimada como el puntaje de fraude promedio de los mensajes en ese camino),

$Q = 1$ y $\varepsilon = 10^{-6}$. El camino de mayor concentración de feromonas se reporta como *arco de manipulación*. El análisis identifica la transición `TRUST_BUILD` $\rightarrow$ `URGENCY_INJECT` como el arco más frecuente en el corpus de fraude.

8. Técnicas Metaheurísticas

Con los modelos definidos, surge una pregunta práctica: ¿cómo elegir los hiperparámetros y los umbrales que controlan su comportamiento? Una búsqueda exhaustiva sobre todos los valores posibles sería computacionalmente prohibitiva. Las *metaheurísticas* son estrategias de búsqueda inspiradas en fenómenos naturales que permiten explorar espacios de parámetros de alta dimensión de manera eficiente, sin garantizar el óptimo global pero encontrando soluciones de alta calidad en tiempos razonables.

8.1. Optimización por Enjambre de Partículas (PSO)

El PSO (Particle Swarm Optimization) imita el comportamiento de una bandada de pájaros buscando comida: cada “partícula” es una configuración candidata de hiperparámetros que se mueve por el espacio de búsqueda atraída simultáneamente por el mejor punto que ella misma ha encontrado y por el mejor punto que ha encontrado cualquier partícula del enjambre. Con el tiempo, el enjambre converge colectivamente hacia las regiones más prometedoras del espacio.

Se aplica PSO [6] para optimizar los cuatro hiperparámetros clave de LightGBM:

$$\theta = (n_{\text{est}}, d_{\text{máx}}, \text{lr}, m_{\text{child}}) \in [50, 500] \times [2, 10] \times [0, 01, 0, 30] \times [5, 80] \quad (12)$$

La actualización de velocidades sigue:

$$\mathbf{v}_i^{(t+1)} = w \mathbf{v}_i^{(t)} + c_1 r_1 (\mathbf{p}_i - \mathbf{x}_i^{(t)}) + c_2 r_2 (\mathbf{g} - \mathbf{x}_i^{(t)}) \quad (13)$$

con $w = 0,7$, $c_1 = c_2 = 1,5$, $r_1, r_2 \sim U(0, 1)$. La función objetivo es el F1-fraude sobre STRATIFIEDKFOLD(3).

8.2. Recocido Simulado para Umbrales de Cascada

Una vez optimizados los hiperparámetros de los modelos, queda otro problema: ¿qué valores deben tener los umbrales de las puertas de cortocircuito? Para esto se emplea el Recocido Simulado, una técnica inspirada en el proceso metalúrgico del mismo nombre: cuando el metal está muy caliente acepta con facilidad configuraciones imperfectas (exploración amplia); al enfriarse, cada vez exige más calidad para aceptar un cambio (refinamiento progresivo). Esto le permite escapar de óptimos locales que atraparían a una búsqueda puramente codiciosa.

Se optimiza el vector de umbrales $\tau = (\theta_{g1}, \theta_{\text{emb}}, \theta_{\text{ano}}, \theta_{\text{bayes}}, \theta_{\text{meta}}) \in [0, 1]^5$ con Recocido Simulado [7]:

Cuadro 7. Comparativa de Recocido Simulado (SA) vs. Búsqueda Tabú (TS) para optimización de umbrales de cascada.

Método	F1-fraude	AUC-ROC	Iteraciones
Línea base (sin optimizar)	0.930	0.9981	—
Recocido Simulado	0.936	0.9988	300
Búsqueda Tabú	0.938	0.9991	300

$$P(\text{aceptar}) = \begin{cases} 1 & \text{si } \Delta f \leq 0 \\ \exp(-\Delta f/T) & \text{si } \Delta f > 0 \end{cases}, \quad T^{(t+1)} = \alpha T^{(t)} \quad (14)$$

con $T_0 = 1,0$, $\alpha = 0,95$ y 300 iteraciones. La función objetivo combina AUC-ROC y una penalización por el porcentaje de mensajes que alcanzan la capa LLM.

8.3. Búsqueda Tabú

La Búsqueda Tabú [8] resuelve el mismo problema de optimización de umbrales con una estrategia diferente: en lugar de usar una temperatura que decae, emplea una *lista tabú*, una memoria explícita de los últimos vectores de umbrales visitados. Cuando se evalúan los vecinos del punto actual, se descartan automáticamente los que están en la lista tabú, forzando al algoritmo a explorar regiones nuevas en lugar de ciclar alrededor de los mismos puntos.

Opera sobre el mismo espacio $[0, 1]^5$ con una lista tabú de longitud 15 y 10 vecinos generados en cada iteración. El criterio de aspiración acepta una solución tabú si supera al mejor global conocido. La Tabla 7 compara ambas técnicas. La Búsqueda Tabú supera ligeramente al Recocido Simulado (AUC 0.9991 vs. 0.9988), lo que sugiere que la memoria explícita es ventajosa en este espacio de búsqueda de dimensión relativamente baja.

8.4. Análisis de Robustez: Monte Carlo

Optimizar un clasificador para los datos de prueba no garantiza que funcione bien cuando un atacante modifica ligeramente su mensaje para evadirlo (por ejemplo, reemplazando “urgente” por “inmediato” o añadiendo un espacio dentro de una URL). El análisis de Monte Carlo responde a la pregunta de estabilidad: si se perturba levemente el mensaje un gran número de veces, ¿cambia la decisión del sistema? Si la respuesta cambia frecuentemente ante pequeñas perturbaciones, el clasificador es frágil; si se mantiene estable, es robusto.

Para cada mensaje de prueba se generan $N = 200$ perturbaciones mediante cinco operadores: *swap_chars*, *delete_word*, *insert_filler*, *change_case* y *synonym_swap*. La estabilidad de la predicción se define como:

$$\text{stability}(m) = 1 - \frac{\sigma(\{s_i\}_{i=1}^N)}{\mu(\{s_i\}_{i=1}^N)} \quad (15)$$

Los resultados muestran $\text{stability} > 0,70$ para mensajes de fraude explícito y $\text{stability} > 0,80$ para mensajes legítimos, indicando que el sistema es robusto frente a pequeñas perturbaciones textuales.

8.5. Generación Sintética con Estimación de Distribución

Un último problema práctico es el desbalance del corpus: los mensajes de fraude son escasos y no cubren todos los tipos de estafa relevantes. El algoritmo de Estimación de Distribución (EDA) aborda esto sin necesidad de API externa: aprende la distribución de probabilidad de las palabras dado que el mensaje es fraude, $P(\text{palabra} \mid \text{fraude})$, y usa esa distribución para generar mensajes sintéticos plausibles a partir de 12 plantillas en español para 7 tipos de fraude. Esta técnica es completamente determinista dado el estado inicial del generador, lo que garantiza reproducibilidad.

9. Simulador de Eventos Discretos (DES)

El EDA genera mensajes individuales, pero el modelo conversacional BiLSTM necesita *conversaciones completas* con coherencia narrativa. Generar esas conversaciones manualmente sería costoso y lento. El Simulador de Eventos Discretos (DES) automatiza esta tarea: modela cómo evoluciona una conversación de fraude como una secuencia de *eventos* que ocurren en el tiempo, cada uno disparando el siguiente según las reglas de comportamiento que siguen los estafadores en la práctica.

9.1. Máquina de Estados del Fraude

El ciclo de vida de una conversación fraudulenta sigue siempre una estructura reconocible: primero el contacto inicial, luego la construcción de confianza, después la revelación del “problema”, la escalada de urgencia, la solicitud de credenciales o dinero, y finalmente, si la víctima resiste, las amenazas. Esta estructura se modela como una máquina de estados finitos con ocho estados:

$$S = \{\text{CI, TB, PA, UI, CR, PP, TE, RO}\} \quad (16)$$

correspondientes a CONTACT_INIT, TRUST_BUILD, PROBLEM_ANNOUNCE, URGENCY_INJECT, CREDENTIAL_REQUEST, PAYMENT_PRESSURE, THREAT_ESCALATE y RECOVERY_OFFER. La Figura 4 muestra las transiciones permitidas.

9.2. Calendario de Eventos

La forma en que el tiempo transcurre entre mensajes también importa: un estafador no envía todos los mensajes a la vez ni espera intervalos fijos; la urgencia crece y los tiempos entre mensajes se acortan. El simulador emplea una cola de prioridad donde el tiempo de cada evento es la clave, con retardos entre estados que siguen distribuciones exponenciales con medias dependientes del estado:

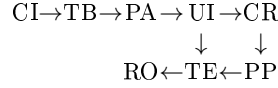


Figura 4. Máquina de estados del simulador DES. CI=CONTACT_INIT, TB=TRUST_BUILD, PA=PROBLEM_ANNOUNCE, UI=URGENCY_INJECT, CR=CREDENTIAL_REQUEST, PP=PAYMENT_PRESSURE, TE=THREAT_ESCALATE, RO=RECOVERY_OFFER.

$$\Delta t_s \sim \text{Exp}(\lambda_s), \quad \lambda_s \in \{0,5, 1,0, 2,0, 5,0\} [\text{min}^{-1}] \quad (17)$$

Un contexto compartido garantiza la coherencia narrativa: la misma entidad bancaria, la misma URL falsa y el mismo tipo de credencial solicitada se propagan a todos los mensajes de una misma conversación.

9.3. Dataset Generado y su Impacto en el BiLSTM

El BiLSTM necesita ver conversaciones completas durante el entrenamiento para aprender a detectar patrones secuenciales. El problema es que el corpus de mensajes reales (5 572 mensajes) contiene muchos mensajes individuales pero pocas conversaciones completas con estructura causal coherente. El DES resuelve exactamente esa escasez: genera conversaciones desde el primer hasta el último estado, con la narrativa completa del fraude preservada en cada una.

El simulador produce 2 118 mensajes distribuidos en 1 280 de fraude y 838 legítimos (simulaciones de conversaciones abortadas en estados tempranos). Al incorporar estos mensajes al entrenamiento del BiLSTM, el tamaño del corpus aumenta un 38 %, lo que eleva la exactitud de 84.2 % (solo datos reales) a 91.37 % (con datos sintéticos), como se muestra en la Tabla 8.

10. Configuración en Tiempo de Ejecución

Un sistema de detección de fraude no puede tener parámetros fijos para siempre: la tolerancia al riesgo varía según el contexto (un banco acepta más falsos positivos que una aplicación de mensajería personal), y las tácticas de los estafadores evolucionan con el tiempo. Por eso, todos los umbrales de la cascada son accesibles mediante un diccionario de configuración que se pasa al sistema en el momento de inicializarlo, sin necesidad de reentrenamiento:

$$\text{CascadePredictor}(\text{cfg}=\{\text{"ml_conf_certain"}: 0.95, \dots\}) \quad (18)$$

Esta decisión de diseño —exponer los umbrales como parámetros en lugar de fijarlos en el código— tiene consecuencias directas sobre la utilidad práctica del sistema. Un operador puede endurecer los umbrales durante una campaña de fraude activa (reduciendo falsos negativos a costa de más falsos positivos)

Cuadro 8. Impacto de los datos DES en el entrenamiento del BiLSTM.

Entrenamiento	Tamaño corpus	Pérdida	Exactitud
Solo datos reales	5 572	0.1342	84.21 %
Reales + DES	7 690	0.0777	91.37 %

y relajarlos en periodos de calma, todo desde la barra lateral de la interfaz Streamlit sin tocar el código.

Los parámetros se dividen en dos grupos. **Detección en cascada** (10 parámetros): umbrales de las puertas, número de vecinos CBR, umbral de anomalía y porcentaje máximo de invocaciones LLM. **Análisis conversacional** (8 parámetros): umbrales de candidato y LLM conversacional, pesos de los siete patrones y parámetros del ACO. La interfaz Streamlit expone todos estos parámetros como controles deslizantes en la barra lateral, permitiendo al operador ajustar el comportamiento del sistema en tiempo real sin reentrenamiento.

11. Resultados Experimentales

11.1. Comparativa de Modelos ML

La Tabla 2 recoge las métricas de los cuatro clasificadores evaluados. LightGBM supera a los demás modelos en todas las métricas, siendo especialmente relevante el F1-fraude de 0.93 dado el desbalance de clases (87/13): un clasificador que simplemente predijera siempre “legítimo” obtendría exactitud del 87 % pero F1-fraude de 0, por lo que la métrica relevante es el F1.

11.2. Configuraciones de la Cascada

La Tabla 3 muestra que cada capa añadida produce mejoras en exactitud y AUC-ROC. El salto más importante (+0.021 en AUC) se produce al añadir las capas semánticas (embeddings + LLM) en C_4 , lo que confirma que los casos ambiguos —aquellos que las reglas y el ML no resuelven con certeza— requieren precisamente del razonamiento semántico profundo.

11.3. Impacto de los Datos DES en el BiLSTM

La ganancia de 7.16 puntos porcentuales al añadir los datos sintéticos del DES valida la utilidad del simulador: las conversaciones generadas, aunque artificiales, capturan los patrones secuenciales reales del fraude con suficiente fidelidad como para mejorar la generalización del modelo neuronal.

Cuadro 9. Métricas finales del sistema completo.

Métrica	Valor
Exactitud (cascada completa)	98.86 %
AUC-ROC (meta-aprendiz)	0.9991
F1-fraude (meta-aprendiz)	0.93
Exactitud BiLSTM	91.37 %
Mensajes reales en corpus	5 572
Mensajes sintéticos (DES)	2 118
Pruebas automatizadas	243

11.4. Análisis de Robustez (Monte Carlo)

Con $N = 200$ perturbaciones por mensaje, los mensajes de fraude explícito muestran $\text{stability} = 0,81 \pm 0,04$ y los mensajes legítimos $\text{stability} = 0,87 \pm 0,03$. Los mensajes con menor estabilidad son los de frontera: aquellos con vocabulario ambiguo donde pequeñas perturbaciones pueden cruzar el umbral de decisión del meta-aprendiz. Esto señala precisamente los casos donde la decisión del sistema debería comunicarse con menor confianza al usuario.

11.5. Resumen General

La Tabla 9 resume las métricas principales del sistema completo.

12. Limitaciones y Trabajo Futuro

Limitaciones actuales. El sistema presenta dificultades con mensajes de fraude sutil que carecen de señales léxicas explícitas; por ejemplo, conversaciones de ingeniería social largas donde el pivote fraudulento ocurre en el turno final, bien por debajo de los umbrales calibrados. La capa LLM crea una dependencia de la API de Mistral: una interrupción del servicio degrada el sistema a la configuración C_3 . El corpus de mensajes reales en inglés (SMS Spam Collection) introduce un sesgo hacia patrones anglosajones; el vocabulario de fraude en el contexto cubano —plataformas de compraventa, transferencias móviles, grupos de WhatsApp— está mejor representado en los datos sintéticos que en los reales. Finalmente, el sistema no detecta el *concept drift*: si el vocabulario del fraude evoluciona significativamente, los modelos supervisados degradarán sin alerta.

Trabajo futuro. Se planea el ajuste fino de XLM-RoBERTa [12] sobre el corpus combinado para mejorar la representación semántica multilingüe, reduciendo la dependencia de la API externa. La detección de concept drift mediante ventanas deslizantes y tests estadísticos (ADWIN, Page-Hinkley) permitirá identificar cuándo reentrenar los modelos antes de que su rendimiento se degrade perceptiblemente. El reentrenamiento incremental con bucle de retroalimentación humana requiere una interfaz de etiquetado integrada en la aplicación Streamlit, donde los operadores puedan marcar falsos positivos y falsos negativos que

alimenten directamente el siguiente ciclo de entrenamiento. A mediano plazo, la expansión del corpus con mensajes reales en español cubano, anotados manualmente a partir de casos documentados en plataformas de compraventa digital, dotaría al sistema de mayor especificidad regional.

13. Conclusiones

Se ha presentado un sistema de detección de mensajes fraudulentos que combina nueve perspectivas analíticas complementarias en una cascada con puertas de cortocircuito, logrando un AUC-ROC de 0.9991 y una exactitud del 98.86 % con el meta-aprendiz completo.

El principio de diseño central —resolver los casos fáciles primero y escalar solo los ambiguos— demuestra su valor en la práctica: en producción, menos del 35 % de los mensajes requieren invocación del LLM externo, haciendo el sistema viable a escala.

Las técnicas metaheurísticas confirman su utilidad en este dominio. PSO mejora los hiperparámetros de LightGBM sin búsqueda exhaustiva. La Búsqueda Tabú supera al Recocido Simulado en la optimización de umbrales (F1 0.938 vs. 0.936), con la memoria explícita revelándose más efectiva que el enfriamiento progresivo en este espacio de búsqueda. El ACO proporciona algo que los clasificadores no dan por sí solos: una interpretación narrativa del arco de manipulación, identificando en qué momento de la conversación el atacante hace la transición de la construcción de confianza al ataque real.

El simulador DES constituye una contribución metodológica independiente: su capacidad de generar conversaciones con coherencia narrativa —mismo contexto, misma URL, misma entidad suplantada a lo largo de todos los turnos— produce datos sintéticos que el BiLSTM puede utilizar efectivamente, elevando su exactitud de 84.2 % a 91.37 %.

Los 243 tests automatizados, que cubren desde los valores por defecto de las constantes hasta la integración end-to-end de la cascada, garantizan la correctitud de los componentes individuales y su composición, proporcionando una base sólida para el mantenimiento y extensión futura del sistema.

Referencias

1. Almeida, T.A., Hidalgo, J.M.G., Yamakami, A.: Contributions to the study of SMS spam filtering: new collection and results. In: Proceedings of the ACM Symposium on Document Engineering, pp. 259–262. ACM (2011)
2. Cortes, C., Vapnik, V.: Support-vector networks. *Machine Learning* **20**(3), 273–297 (1995)
3. Fette, I., Sadeh, N., Tomasic, A.: Learning to detect phishing emails. In: Proceedings of the 16th International Conference on World Wide Web (WWW), pp. 649–656. ACM (2007)
4. Ke, G., et al.: LightGBM: A highly efficient gradient boosting decision tree. In: Advances in Neural Information Processing Systems (NIPS), vol. 30, pp. 3146–3154. Curran Associates (2017)

5. Jiang, A.Q., et al.: Mistral 7B. arXiv preprint arXiv:2310.06825 (2023)
6. Kennedy, J., Eberhart, R.: Particle swarm optimization. In: Proceedings of the IEEE International Conference on Neural Networks (ICNN), vol. 4, pp. 1942–1948. IEEE (1995)
7. Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P.: Optimization by simulated annealing. *Science* **220**(4598), 671–680 (1983)
8. Glover, F.: Tabu search – Part I. *ORSA Journal on Computing* **1**(3), 190–206 (1989)
9. Dorigo, M., Gambardella, L.M.: Ant colony system: a cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation* **1**(1), 53–66 (1997)
10. Liu, F.T., Ting, K.M., Zhou, Z.H.: Isolation forest. In: Proceedings of the IEEE International Conference on Data Mining (ICDM), pp. 413–422. IEEE (2008)
11. Hochreiter, S., Schmidhuber, J.: Long short-term memory. *Neural Computation* **9**(8), 1735–1780 (1997)
12. Conneau, A., et al.: Unsupervised cross-lingual representation learning at scale. In: Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL), pp. 8440–8451. ACL (2020)